

WISE-Fix: Supplementary Material

1 Verifier Synthesis, Rule Semantics, and Reproduction Materials

This supplement expands Section 2.2 by specifying the synthesis interface, training-example policy, rule representation, operator boundaries, evidence contracts, and executable examples. The accompanying package pins `wise-fix-reference-3.1`. Its two candidate suites and synthetic fixtures support reproducible checks of the reference workflow; they are not the original benchmark-accepted suite collection. The system prompt reproduced below matches the inspected reference adapter, not a recovered historical experiment prompt. Exact reproduction of the reported experiments additionally requires the original suites, data membership, CWE specifications, and evaluated settings.

1.1 Inputs, Example Selection, and Leakage Controls

For weakness w , synthesis consumes its specification d_w , labeled training transitions, the fixed typed operator library $\mathcal{P}$, and rule interface Σ . Each transition is $x = (S^-, \Delta, S^+)$, where Δ is relative to a selected actual parent. Source-state roles and correspondence are retained through extraction and execution. Training and development identities are disjoint; test instances are excluded from synthesis, revision, and artifact selection.

Non-security labels supply negative example roles but do not establish weakness absence. The reference selector does not independently confirm that these rows fail to repair w . This differs from the confirmed target-relative negatives specified in Section 2.2; alignment requires documented target-relative screening before example selection.

Example prompts contain the role **positive** or **negative**, pre/post source text, and the diff. Instance identifiers, repository identifiers, benchmark CVE/CWE annotations, advisories, author metadata, and commit descriptions are not interpolated as example metadata. Code identifiers remain. Upstream preparation must also prevent annotation leakage embedded in source comments or text.

The CWE specification and its version must be archived. Accepted-artifact manifests record synthesis and scorer membership, construction identities, partition digests, and the accepted revision. Reference policies and defaults must not be presented as recovered experimental settings.

1.2 Synthesis Prompt and Executed Request

The following interface summarizes the intended synthesis task. The engine functions `llm_generate()` and `synthesis_schema()`, together with the registered operator contracts, define the reference request structure. These definitions are distinct from the complete serialized requests of an actual run.

ROLE: Compose weakness-specific executable obligations from a fixed program-analysis library.

INPUT: Target weakness w , specification d_w , labeled training transitions, typed operators $\mathcal{P}$, and rule interface Σ .

Table 1: Example-selection policies and default limits of the reference runtime.

Aspect	Reference policy
Positive membership	Security-labeled rows whose <code>cwes</code> list contains <code>w</code> .
Negative membership	All non-security-labeled rows in the corresponding partition; target-relative screening is not implemented by the selector.
Selection	At most 20 training examples per class, ordered by supplied <code>timestamp</code> , then <code>id</code> . Missing timestamps use zero; chronological interpretation requires correctly populated timestamps.
Multiple CWEs	A positive may participate in each explicitly listed suite; membership is recorded.
Missing annotations	A security row without <code>w</code> is excluded from that suite’s construction corpus, but not from the complete test partition.
Hierarchy	No implicit ancestor mapping; any benchmark mapping must be applied and documented upstream.
Development	All eligible development rows; both classes are required. Development examples are excluded from initial synthesis.
Source limits	Default limits are 8,000 characters per source state and 16,000 diff characters. Other extraction and enumeration limits are recorded in configuration.
Scorer corpus	All eligible training rows, irrespective of verdict; development data select regularization and stopping point.

TASK: Construct linked Precondition, Repair, and Safety obligations. Specify selector scopes, predicate arguments, evidence requirements, and compatible shared bindings. Represent alternative repair mechanisms separately.

OUTPUT: Return one schema-conforming JSON suite whose compiled execution reports SATISFIED, VIOLATED, or UNRESOLVED, with supporting, contradictory, or missing evidence.

CONSTRAINTS: Compose only permitted operators; do not add or modify primitives. Use code-derived runtime evidence and exclude labels and identifying metadata from predicates. Invoke no LLM during execution and leave final verdict assignment to the frozen decision module.

The exact reference system message is:

Synthesize a CWE-specific executable suite using ONLY the fixed operators supplied. Return one JSON object conforming to the schema example. Mandatory predicates are conjoined within a witness, alternatives are existential, and bindings must remain compatible. No arbitrary code, literals from identifying metadata, new operators, or runtime LLM calls. Each repair alternative must include an explicit edit predicate marked `attribution=true`.

Listing 1: System message used by the reference synthesis adapter.

The user message serializes `examples`, `operators`, `semantic_contracts`, `schema`, `diagnostics`, `previous`, `cwe`, and `cwe_specification`. Initial generation uses empty diagnostics and no previous suite; revisions include development diagnostics and the previous candidate.

Requests use the configured model, temperature zero, and JSON-object response mode. Temperature zero does not guarantee identical responses from a hosted model. Reproduction therefore requires preservation of the actual requests, responses, provider model identifier, usage records, configuration, and accepted JSON. Mocked requests, synthetic inputs, and placeholder specifications must be identified as interface demonstrations rather than live synthesis logs.

1.3 Rule Representation and Semantic Checks

A reference suite contains `schema`, `cwe`, and one or more `alternatives`. Each alternative declares an identifier, typed pre/post AST selectors, and nonempty `pre`, `repair`, and `safety`

predicate lists. Predicates have unique identifiers, fixed operator names, and operator-specific arguments referencing selector bindings. All listed stage predicates are mandatory; aggregation is fixed by the runtime.

The engine’s `synthesis_schema()` defines the example-based interface supplied to the LLM. Any separately exported structural JSON Schema is a companion specification rather than a recovered experimental artifact. Semantic validation by `validate_suite()` remains authoritative: bindings must exist, node types and source-state roles must match, Precondition uses pre-state evidence, Safety uses post-state evidence, and Repair requires an explicit edit and a qualifying mitigation predicate marked for attribution. No arbitrary Python or metadata predicate is executed.

Optional rejection rules refer to existing predicate outcomes within one repair alternative. Their conditions must be decisive and include a `VIOLATED` condition; unresolved facts or metadata alone cannot activate rejection. Complete candidate examples are provided in `Verifiers/suites/CWE-125.json` and `Verifiers/suites/CWE-119.json`.

1.4 Fixed Operators and Analysis Boundaries

The inspected engine registers 22 operator signatures in `OPS` and eight specialized profile contracts in `MANUSCRIPT_CONTRACTS`. The pinned engine, including its helper functions and reporting semantics, defines executable reference behavior. Operator names alone are insufficient to reproduce their semantics.

Table 2: Operator families and their evidence boundaries.

Family	Local evidence and limitations
AST inspection	Parsed node kinds, names, references, operators, containment, and same-function relationships; textual resemblance is insufficient.
CFG checks	Dominance and exiting guard branches within supported intrafunction flow. Unsupported or unavailable flow preserves uncertainty.
Correspondence and edits	Mapped elements and explicit parent-relative changes. Disappearance alone is not repair evidence; ambiguous correspondence remains unresolved.
Destination-copy profile	Guard mismatch, linked guard correction, counted <code>memcpy</code> loop, and matching destination increment. Does not establish all source-read bounds or arithmetic safety.
Length-normalization profile	Direct conversion versus inserted <code>unsigned char</code> normalization, linked local definition, arithmetic update, and <code>ReadBlob</code> size use. Ambiguous writes preserve uncertainty.

The reference parser accepts supported self-contained source with available type declarations. It is not a build-aware preprocessor or a general interprocedural analysis backend. Preprocessor directives, unavailable facts, oversized contexts, exhausted enumeration, and unsupported control flow may prevent required structural evidence.

Complete scoped enumeration means completeness within the configured extraction and analysis scope, not across all repository behavior. The operators do not establish unrestricted memory safety, allocated object extents, overflow freedom, or whole-program functional correctness.

1.5 Three-Valued Aggregation and Linked Witnesses

For a nonempty set of mandatory predicate outcomes $q_1, \dots, q_m$ within one witness,

$$\text{Conj}(q_1, \dots, q_m) = \begin{cases} V, & \text{if any } q_i = V, \\ S, & \text{if all } q_i = S, \\ U, & \text{otherwise,} \end{cases}$$

where S, V, and U denote SATISFIED, VIOLATED, and UNRESOLVED.

Across candidate witnesses and repair alternatives, a stage is SATISFIED when a witness satisfies all mandatory predicates of that stage. It is VIOLATED only when enumeration is complete within scope and every candidate is decisively violated. The reference runtime also treats complete enumeration with no match for an existential requirement as VIOLATED. Otherwise, the outcome is UNRESOLVED. Partial enumeration may establish satisfaction through a supporting witness but cannot establish exhaustive exclusion.

Selectors retain candidate sets and completeness flags. Shared bindings and evidence remain available for compatibility checking. Stage-wise satisfaction through unrelated witnesses is insufficient: final acceptance requires compatible support across Precondition, Repair, and Safety within the same repair alternative.

1.6 Readable Executable Profiles

The following notation summarizes the shipped JSON profiles; it does not introduce independent implementations or new analysis primitives. The destination profile retains the manuscript's benchmark CWE-125 mapping while checking its specific encoded destination-copy property.

```
Selectors:
  old_guard: pre If
  old_write: pre Assignment
  new_guard: post If
  new_write: post FuncCall named memcpy

Precondition:
  DESTINATION_GUARD_MISMATCH(old_guard, old_write)

Repair:
  DESTINATION_REPAIR_LINK(
    old_guard, new_guard, old_write, new_write)
  EDIT_DESTINATION_GUARD(
    old_guard, new_guard, new_write),
    attribution=true

Safety:
  DESTINATION_COPY_GUARDED(new_guard, new_write)

Shared profile parameters:
  destination=tga->bitmap
  source=decompression_buffer
  position=bitmap_caret
  source_position=buffer_caret
  count=encoded_pixels
  width=pixel_block_size
  capacity=image_block_size
  iterator=i
  inner_iterator=j
```

Listing 2: Reference destination-copy repair alternative.

The pre-patch witness links the existing source-position check to the selected indexed destination write. Repair links that guard/write pair to the changed destination-position guard and post-patch copy, with explicit edit evidence. Safety checks the exiting destination-total guard, dominance, supported counted loop, copy width, and matching destination increment.

This profile explicitly models replacement of indexed copying with `memcpy`. An indexed-read guard-insertion branch would require a separate alternative and corresponding operator support.

```

Selectors:
  old_length, old_arithmetic: pre Assignment
  old_sink: pre FuncCall named ReadBlob
  new_length, arithmetic: post Assignment
  sink: post FuncCall named ReadBlob

Precondition:
  DIRECT_LENGTH_FLOW(
    old_length, old_arithmetic, old_sink)

Repair:
  EDIT_LENGTH_NORMALIZES(
    old_length, new_length),
  attribution=true

Safety:
  NORMALIZED_LENGTH_FLOW(
    new_length, arithmetic, sink)

Shared profile parameters:
  variable=length
  accumulator=combined_length
  producer=ReadBlobByte
  consumer=ReadBlob
  image=image
  outer_type=MagickSizeType
  size_type=size_t
  normalization_type=unsigned char

```

Listing 3: Reference unsigned-length-normalization alternative.

The normalization profile links the original producer-to-size conversion, inserted unsigned-byte conversion, accumulator use, and postincrement size argument within the same local definition/use chain. An unused cast or a normalized unrelated variable does not satisfy this alternative.

1.7 Source Examples, Controlled Fixtures, and Trace Evidence

The supplied manuscript associates the libgd and ImageMagick changes with CVE-2016-6905 and CVE-2017-5511, respectively. Historical reproduction requires the repository URL, full commit and selected-parent hashes, source files, and actual diff. These historical identities have not been independently verified here. The runnable fixtures add illustrative declarations and function bodies; they are synthetic contexts rather than historical repository snapshots.

In the libgd fragment, the old check uses `buffer_caret`; the new check uses `bitmap_caret`, followed by `memcpy` and a destination increment. Source and destination bounds are distinct obligations. The additional reported edit from `& 127` to `&!TGA_RLE_FLAG` must be preserved in the historical diff and examined independently; the destination profile does not establish its equivalence or correctness.

In the ImageMagick fragment, the inserted `unsigned char` conversion precedes the outer size cast and subsequent uses. Neither example establishes complete program safety.

Each evidence atom records its operator, predicate, stage, alternative, outcome, polarity, argument bindings, and reason. Available bound facts carry source-state roles, locations, and source hashes. Unresolved atoms may instead identify missing bindings or analysis requirements.

Stable provenance keys avoid counting replay-induced line shifts as new support. The validator checks membership, provenance, scope, correspondence, completeness, consistency, and witness compatibility; it does not rerun weakness predicates. Within a suite, validated seed-applicable rejection takes precedence. Missing acceptance evidence does not cancel an independently supported rejection. Otherwise, unsupported acceptance yields INCONCLUSIVE, rather

Table 3: Behavior checked by controlled reference tests.

Controlled condition	Required interpretation
Linked destination-bound copy or length normalization	VERIFIED only with mandatory obligations, compatible witnesses, and valid evidence.
Empty failure branch, wrong copy width, or unrelated cast	Cannot be VERIFIED; rejection versus uncertainty depends on executed evidence and completeness checks.
Unsupported flow or intervening length assignment	Required structural or definition/use evidence remains unresolved.
Missing supporting evidence or forged diff	Cannot establish acceptance; invalid evidence alone does not establish contradiction.
Unrelated additional code change	Does not independently reject an otherwise supported repair witness.

than an invented negative fact.

For composite units, actual post-state Safety confirmation and qualifying seed mitigation are additional acceptance requirements. A contradicted search extension alone does not reject the seed. Across suites, any VERIFIED result verifies the seed; otherwise, any INCONCLUSIVE result, or no applicable suite, yields INCONCLUSIVE; remaining cases are REJECTED. Only VERIFIED seeds are ranked. Ranking does not change verdicts or full-set binary metrics.

1.8 Validation, Freezing, and Reproduction

Candidate suites undergo schema, type, and operator checks, smoke testing, and development validation with bounded revision. Accepted artifacts are frozen with their configuration, operator contracts, fitted scorer, and compatible runtime version. Test annotations do not guide artifact selection, and online execution performs no LLM inference or rule adaptation.

The accompanying replication package documents dependencies, input formats, configuration, and execution commands. Its illustrative reference profiles are distinct from the original benchmark-accepted artifacts; reproducing the reported results requires the original artifacts, dataset manifests, and evaluated settings.